Course 2 Master Revision: Building RAG Applications

Building Production RAG Systems with LangChain & LlamaIndex

1. Fundamentals of Retrieval-Augmented Generation (RAG)

Core Problems Solved by RAG

- **Out-of-Date Knowledge:** LLMs are frozen at training time. RAG continuously retrieves fresh external data without model retraining.
- **Hallucinations & Attribution:** Base LLMs generate text without citations. RAG forces responses grounded strictly in retrieved context.
- **Data Privacy & Security:** Private enterprise documents remain in secure vector stores and are retrieved based on user permissions.

VISUAL ARCHITECTURE: END-TO-END 5-STAGE RAG PIPELINE

The 3-Part Augmented Prompt Structure

- **1. System Instruction:** 'Answer strictly using the provided context below. If unknown, say I do not know.'
- **2. Retrieved Context:** The exact text chunks fetched from ChromaDB / Vector Store concatenated into one string.
- **3. User Question:** The original prompt or query entered by the user.

2. Vector Operations, Distance Metrics & Inference Parameters

Similarity Distance Metrics (Interview Core)

Similarity Metric	Mathematical Formula	When to Use in Production
Cosine Similarity	cos(theta) = (A . B) / (||A|| ||B||)	Measures angular direction. Best when text chunk lengths vary significantly across dataset.
Dot Product	A . B = sum(A_i B_i)	Measures magnitude + direction. Fast to compute; identical to Cosine when vectors are L2-normalized.
Euclidean (L2) Distance	||A - B|| = sqrt(sum((A_i - B_i)^2))	Measures spatial endpoint distance. Smaller distance value indicates higher semantic similarity.

LLM Decoding Strategies & Hyperparameters

- **Greedy Decoding:** Always selects the #1 highest-probability token. 100% deterministic (same prompt = same response). Ideal for factual RAG, data extraction, and code generation.
- **Sampling Decoding:** Randomly samples tokens based on a probability distribution. Unlocks Temperature, Top-P, and Top-K.
- **Temperature (0.0 - 2.0):** Low temperature (0.1-0.3) keeps output factual; high temperature (0.8-1.5) increases randomness & creativity.
- **Top-P (Nucleus Sampling):** Restricts token choices to the top cumulative probability threshold (e.g., top 90% most likely words).
- **Top-K Sampling:** Restricts token selection strictly to the top K most probable next words.

SYSTEM DESIGN INTERVIEW TIP: GREEDY VS SAMPLING

Always recommend Greedy Decoding (Temperature=0) for Enterprise RAG extraction & financial reporting to guarantee 100% reproducibility!

3. Document Combination & Memory Strategies

The 4 Combination Strategies (chain_type)

- **stuff (95%+ Industry Standard):** Concatenates all retrieved chunks into 1 prompt payload. Requires only 1 API call (fastest & cheapest). Modern 128k+ LLMs make this standard.
- **map_reduce:** Runs an individual LLM call for EACH chunk, then runs a final LLM call to synthesize all answers. (Used for offline 500-page book summarization).
- **refine:** Loops sequentially through chunks, refining the answer step-by-step.
- **map_rerank:** Scores each chunk's answer from 0 to 100 and returns the highest-scoring response.

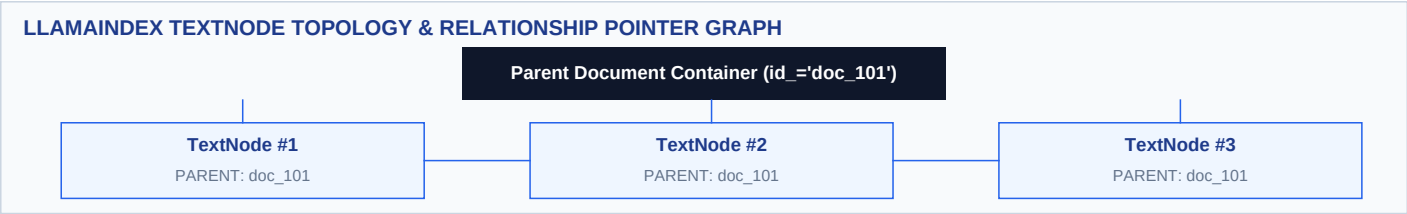
Source Citations & Memory Evolution

- **return_source_documents=True:** Instructs LangChain to return the exact Document chunks retrieved from ChromaDB alongside the final answer for citation UI display.
- **Conversational Memory:** Legacy chains used get_chat_history=lambda h: h. Modern LCEL uses ChatPromptTemplate with MessagesPlaceholder(variable_name='chat_history').

4. LangChain vs. LlamaIndex Deep Architecture & Abstractions

Framework Feature Mapping

Component / Task	LangChain Implementation	LlamaIndex Implementation	Production Advantage
Raw Container	Document(page_content, metadata)	Document(text, metadata, relationships, id_)	LlamaIndex tracks parent/child graph pointers natively.
Text Chunk	Document chunk	TextNode (or Node)	LlamaIndex nodes store explicit sibling & parent IDs.
Directory Loader	DirectoryLoader	SimpleDirectoryReader	LlamaIndex natively parses PDF, CSV, TXT, HTML in 1 call.
Text Splitter	RecursiveCharacterTextSplitter	SentenceSplitter (NodeParser)	LlamaIndex returns list[TextNode] directly.
All-in-One RAG	RetrievalQA / LCEL chain	index.as_query_engine()	LlamaIndex embeds, retrieves & queries in 1 line.
Memory RAG	ConversationalRetrievalChain	index.as_chat_engine()	LlamaIndex chat_engine handles memory automatically.



Response Synthesis Strategies (response_mode)

- **compact (Default / Stuffed):** Stuffs node chunks into prompt. Pre-merges chunks if text approaches context window limit to avoid token overflow crashes.
- **tree_summarize (Map-Reduce):** Builds a tree of LLM responses from chunks and synthesizes a final root summary response.
- **refine:** Sequentially passes Chunk 1 to form Answer 1, then passes (Answer 1 + Chunk 2) to refine answer.
- **simple_summarize / accumulate:** Truncates nodes or applies prompt to each node independently.

Chat Engine Memory Modes (chat_mode)

- **condense_question:** Uses LLM + chat history to rewrite follow-up questions into standalone queries before vector retrieval.
- **context:** Retrieves context nodes for every turn and injects chat history directly into system prompt.
- **react:** Wraps index inside a ReAct agent loop capable of reasoning and deciding when to query.

Document Class Portability & Format Converters

- **Import Swapping Error:** Swapping imports directly throws AttributeError because LangChain uses doc.page_content while LlamaIndex uses doc.text.
- **Official Converters:** LlamaIndex -> LangChain: llama_doc.to_langchain_format() | LangChain -> LlamaIndex: LlamaDocument.from_langchain_format(lc_doc).

Parent-Child Retrieval Database Architecture

- **LangChain (ParentDocumentRetriever):** Requires 2 separate databases (Vector Store for child chunks + external DocStore/Redis for parent docs) managed by an external lookup class.
- **LlamaIndex (TextNode):** Stores graph pointers directly inside node.relationships[NodeRelationship.PARENT]. Serializes relationship pointers into vector DB metadata, requiring ONLY 1 Vector Database.

StorageContext Architecture

- **StorageContext:** Decouples storage from index logic via VectorStore, DocStore, and IndexStore. Allows swapping vector databases without changing downstream query code.

5. Exhaustively Annotated Code Implementations

Code Block 1: Production-Grade LangChain RAG Pipeline (Modern LCEL Standard)

```
# ================================================================================
# PRODUCTION LANGCHAIN LCEL RAG PIPELINE
# (Standard modern implementation using official create_retrieval_chain helpers)
# ================================================================================
# STEP 1: Load raw document into LangChain Document container
# Reads file into memory and returns a list containing 1 Document object (page_content + metadata)
from langchain_community.document_loaders import TextLoader
loader = TextLoader('company_policies.txt', encoding='utf-8')
documents = loader.load()

# STEP 2: Recursive Character Text Splitting
# Recursively splits documents into 500-char chunks with 50-char overlap for context continuity
from langchain_text_splitters import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = text_splitter.split_documents(documents)

# STEP 3: Dense Vector Embedding & Vector Database Ingestion (ChromaDB)
# Converts text chunks into 1536-dimensional dense vectors and stores them in ChromaDB
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vectorstore = Chroma.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={'k': 3})  # Fetches top 3 most relevant chunks

# STEP 4: Construct Augmented System Prompt Template
# IMPORTANT: Prompt MUST define {context} for document stuffing and {input} for user query
from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
('system', 'Answer strictly using context below. If unknown, say NOT FOUND IN CONTEXT:\n\n{context}'),
('human', '{input}')
])

# STEP 5: Assemble Production LCEL RAG Chain
# 1. create_stuff_documents_chain: Concatenates retrieved Document chunks into {context} & invokes LLM
# 2. create_retrieval_chain: Connects vector store retriever to the stuff documents chain
from langchain_openai import ChatOpenAI
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain.chains import create_retrieval_chain
llm = ChatOpenAI(model='gpt-4o', temperature=0.0)
combine_docs_chain = create_stuff_documents_chain(llm, prompt)
production_rag_chain = create_retrieval_chain(retriever, combine_docs_chain)

# STEP 6: Execute Production Chain
# Input key is standardized to {'input': '...'}. Output dict contains ['input', 'context', 'answer']
response = production_rag_chain.invoke({'input': 'What is the eating policy?'})
print(' LLM Answer:', response['answer'])
print(' Retrieved Citation Chunks Count:', len(response['context']))
```

Code Block 2: Complete LlamaIndex Pipeline (ChromaDB + QueryEngine + ChatEngine)

```python
# STEP 1: Global Settings Configuration
from llama_index.core import Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
Settings.llm = OpenAI(model='gpt-4o', temperature=0.0)
Settings.embed_model = OpenAIEmbedding(model='text-embedding-3-small')

# STEP 2: Load Documents & Parse TextNodes
from llama_index.core import SimpleDirectoryReader
from llama_index.core.node_parser import SentenceSplitter
documents = SimpleDirectoryReader(input_dir='./data', recursive=True).load_data()
node_parser = SentenceSplitter(chunk_size=512, chunk_overlap=50)
nodes = node_parser.get_nodes_from_documents(documents)

# STEP 3: Persistent ChromaDB Vector Storage via StorageContext
import chromadb
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext
db = chromadb.PersistentClient(path='./chroma_db_storage')
chroma_collection = db.get_or_create_collection('enterprise_rag')
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# STEP 4: Build VectorStoreIndex (1-Step Embed + Storage)
from llama_index.core import VectorStoreIndex
index = VectorStoreIndex(nodes, storage_context=storage_context)

# STEP 5A: All-in-One Stateless QueryEngine Execution
query_engine = index.as_query_engine(text_qa_template=custom_prompt, similarity_top_k=3, response_mode='compact')
query_response = query_engine.query('What was Apple Q3 revenue?')

# STEP 5B: Stateful ChatEngine Execution (Conversational Memory)
chat_engine = index.as_chat_engine(chat_mode='condense_question', similarity_top_k=3)
chat_turn1 = chat_engine.chat('What was Q3 revenue?')
chat_turn2 = chat_engine.chat('What about Services revenue?')
```

Code Block 3: Framework Portability & Document Conversion

```python
# 1. Instantiate LlamaIndex Document
from llama_index.core import Document as LlamaDocument
llama_doc = LlamaDocument(text='Revenue was $85.8B.', metadata={'year': 2025}, id_='doc_999')

# 2. Convert LlamaIndex -> LangChain Document
lc_doc = llama_doc.to_langchain_format()
print('Converted LangChain page_content:', lc_doc.page_content)

# 3. Convert LangChain -> LlamaIndex Document
new_llama_doc = LlamaDocument.from_langchain_format(lc_doc)
print('Re-converted LlamaIndex text:', new_llama_doc.text)
```

6. Production System Design & LLMOps

Prototyping vs. Production (Gradio vs. FastAPI)

- **Gradio (gr.Interface().launch()):** Designed for 5-minute Python demos and internal manager previews. Lacks security, state persistence, and scalability.
- **FastAPI (Production Standard):** Used to serve LLM agents via REST microservices. Supports async/await, Pydantic validation, and token streaming (StreamingResponse / Server-Sent Events).

LLM Serving Runtime (vLLM)

- **PagedAttention Technology:** Open-source serving engine for hosting Llama 3 / Mistral on GPUs. Uses PagedAttention (virtual memory management for KV-cache) to eliminate memory fragmentation.
- **Performance Impact:** Boosts GPU serving throughput by 2x to 4x and dramatically lowers Time to First Token (TTFT).

Production Caching Strategies

- **Prompt Caching (API-level):** Caches static system prompt prefixes or large RAG context documents on LLM provider servers (Anthropic/OpenAI), cutting input token costs by 50-90%.
- **Semantic Caching (Application-level):** Stores query-answer pairs in a vector cache (Redis VL / GPTCache). If a new query is >95% semantically similar to a past query, returns the cached answer instantly with 0ms LLM latency and $0 API cost.

7. Hands-On Practice Lab References

Lab 1: Dual-Framework RAG Pipeline (dual_framework_rag_lab.py)

- **Location:** coursera_notes/dual_framework_rag_lab.py
- **Part 1 (LangChain):** TextLoader -> CharacterTextSplitter(300, 30) -> Chroma -> RetrievalQA.
- **Part 2 (LlamaIndex):** SimpleDirectoryReader -> VectorStoreIndex -> as_query_engine(response_mode='compact') & as_chat_engine().
- **Part 3 (Portability):** llama_doc.to_langchain_format() format converter practice.

Lab 2: Zero-Boilerplate LangChain Challenge (langchain_scratch_challenge.py)

- **Location:** coursera_notes/langchain_scratch_challenge.py
- **Goal:** Build pure LCEL Pipe (|) RAG chain from scratch with anti-hallucination guardrails and automated test suite.

Lab 3: Zero-Boilerplate LlamaIndex Challenge (llamaindex_scratch_challenge.py)

- **Location:** coursera_notes/llamaindex_scratch_challenge.py
- **Goal:** Build LlamaIndex RAG system from scratch using SimpleDirectoryReader, SentenceSplitter text nodes, QueryEngine & ChatEngine.